

Aula 5 – Triggers

Álvaro G. Pagliari
alvaro.pagliari@unoesc.edu.br

- Revisão Aula Passada
 - Functions e Procedures
- Triggers
- Exercícios





Funções





- Programação estruturada
 - Procedimentos
 - Funções
- Quando utilizamos?
 - Queremos padronizar uma função (que pode ser executada por vários sistemas distintos)
 - Queremos aumentar a segurança (controlando o acesso aos dados e às regras de negócio)





```
CREATE [ OR REPLACE ] FUNCTION name( [ [ argname]
argtype] )
[ RETURNS tipo | [TABLE (cols)] ]
AS $$
[DECLARE var tipo;]
BEGIN
    operações
END;
$$ LANGUAGE plpgsql;
```



- Coalesce
 - Retorna o primeiro valor não nulo na lista

```
COALESCE(value [, ...])
```

- CASE

```
CASE expression  
  WHEN value THEN result  
  [WHEN ...]  
  [ELSE result]  
END
```





- Nullif
 - Retorna null se dois valores são iguais, senão retorna o primeiro valor

```
NULLIF(value1, value2)
```





- Manipulando STRINGS
 - Concatenar
 - ‘Exemplo ’ || **valor** || ‘ de concatenação’
 - concat(s1, s2, ...)
 - concat(var1,var2,’ lol ’, var 3)
 - Pegar o tamanho de uma string
 - lenght(string)
 - Operação com Maiúscula e Minúscula
 - upper(string) – TUDO MAIÚSCULO
 - lower(string) – tudo minúsculo
 - initcap(string) – Primeira Maiúscula





- Manipulando STRINGS
 - Colocar aspas em strings
 - `quote_literal(string)` – Sempre coloca
 - `quote_ident(string)` – Coloca quando necessário
 - `replace(string text, from text, to text)`
 - `replace('abcdefabcdef', 'cd', 'XX') = abXXefabXXef`
 - Remover caracteres
 - `trim([leading | trailing | both] [characters] from string)`
 - `trim(both 'x' from 'xTomxx')`





- Data atual
`now()`
- Extrair partes de uma data
 - `extract(field from timestamp)`
Ex: `extract(hora from '21:12:34')`
- Conta com datas
 - `date '2001-09-28' + interval '1 hour'`
- Calcular idade
 - `age(timestamp, timestamp)`





- Formatar números
 - `round(valor, numero)`
 - Ex: `round(2.3456, 2)`
- Gerar números aleatórios
 - `random();`



Triggers



Triggers

- Gatilhos (triggers) são objetos acessórios a tabelas e visões que funcionam como “ouvidores” (listeners) de eventos.
- O objetivo da criação de triggers é observar ocorrências de inserção, atualização e exclusão de registros ou execução de comandos SQL sobre os objetos aos quais os gatilhos estão vinculados, ANTES ou DEPOIS da sua ocorrência.
- É comum que sejam implementados para executar operações que são derivadas, diretamente, de outras operações.
- Ex.: gravar logs de manutenção de dados

Triggers



```
CREATE [CONSTRAINT] TRIGGER name {BEFORE | AFTER | INSTEAD  
OF} {event [OR...]}  
    ON table_name  
    [FROM referenced_table_name]  
    [NOT DEFERRABLE | [DEFERRABLE] {INITIALLY IMMEDIATE |  
INITIALLY DEFERRED}]  
    [ REFERENCING { { OLD | NEW } TABLE [ AS ]  
transition_relation_name } [ ... ] ]  
    [FOR [EACH] {ROW | STATEMENT}]  
    [WHEN (condition)]  
    EXECUTE PROCEDURE function_name(arguments)
```

Triggers

- CONSTRAINT
 - Somente pode ser usado em triggers do tipo AFTER ROW, permite definir o *timing* de execução da trigger.
- Momento da Execução
 - BEFORE – Antes
 - AFTER – Depois
 - INSTEAD OF – Ao invés de
- event
 - INSERT
 - DELETE
 - UPDATE
 - FOR columns
 - TRUNCATE



Triggers

- Timing
 - NOT DEFERRABLE | DEFERRABLE
 - Indica se a execução da trigger pode ser “adiada” ou não. Quando uma trigger é adiada, significa que ela poderá ser executada no final da transação e não após o evento definido.
 - INITIALLY IMMEDIATE | INITIALLY DEFERRED
 - Se a trigger for DEFERRABLE então é possível selecionar qual o timing padrão de execução. Se for selecionado o INITIALLY IMMEDIATE então a trigger deve ser executada após cada statement. Se a opção selecionada for INITIALLY DEFERRED então a trigger será executada apenas ao final da transação.
- FOR
 - EACH ROW
 - Especifica que para cada linha será executada a trigger
 - EACH STATEMENT
 - Especifica que a trigger somente será executada uma vez





- REFERENCING
 - Possibilita o acesso as tabelas OLD e NEW quando a trigger é do tipo FOR EACH STATEMENT
- [WHEN (condition)]
 - Permite estabelecer uma condição para a execução da ação do trigger
 - Exemplo: executar apenas se o registro do funcionário indicar que o mesmo está ativo no cadastro
- EXECUTE PROCEDURE function_name(arguments)
 - Define qual função implementa a ação do trigger
 - Funções que atendem gatilhos são conhecidas como **trigger functions**
 - São **diferentes** de **funções comuns** por retornarem **trigger**



- Exemplo

```
CREATE TRIGGER check_update  
BEFORE UPDATE ON accounts  
FOR EACH ROW  
WHEN (OLD.balance IS DISTINCT FROM NEW.balance)  
EXECUTE PROCEDURE check_account_update();
```

Triggers

- Exemplo 2

```
CREATE OR REPLACE FUNCTION registra_log() RETURNS TRIGGER AS $body$  
    DECLARE dados_antigos TEXT; dados_novos TEXT;  
BEGIN  
    IF (TG_OP = 'UPDATE') THEN  
        dados_antigos := ROW(OLD.*);  
        dados_novos   := ROW(NEW.*);  
        INSERT INTO log VALUES (dados_antigos, dados_novos);  
        RETURN NEW;  
    ELSEIF (TG_OP = 'DELETE') THEN  
        dados_antigos := ROW(OLD.*);  
        INSERT INTO log VALUES (dados_antigos, DEFAULT);  
        RETURN OLD;  
    ELSEIF (TG_OP = 'INSERT') THEN  
        dados_novos := ROW(NEW.*);  
        INSERT INTO log VALUES (DEFAULT, dados_novos);  
        RETURN NEW;  
    end if;  
END;  
$body$  
language plpgsql;
```

Triggers

- Exemplo 2

```
CREATE OR REPLACE TRIGGER log_hospede  
AFTER INSERT OR UPDATE OR DELETE ON hospedes  
FOR EACH ROW EXECUTE PROCEDURE registra_log();
```

Triggers



When	Event	Row-level	Statement-level
BEFORE	INSERT/UPDATE/DELETE	Tables and foreign tables	Tables, views, and foreign tables
	TRUNCATE	—	Tables
AFTER	INSERT/UPDATE/DELETE	Tables and foreign tables	Tables, views, and foreign tables
	TRUNCATE	—	Tables
INSTEAD OF	INSERT/UPDATE/DELETE	Views	—
	TRUNCATE	—	—



Triggers

- Variáveis Especiais
 - NEW: Record que guarda os novos dados da linha que está sendo inserida ou atualizada em operações row-level. Nula quando for delete ou statement-level.
 - OLD: Record que guarda os dados velhos da linha que está sendo atualizada ou excluída em operações row-level. Nula quando for inserção ou statement-level.
 - TG_NAME: Contêm o nome da trigger que está sendo executada.
 - TG_WHEN: Contêm uma string de quando a trigger está sendo executada BEFORE, AFTER ou INSTEAD OF
 - TG_LEVEL: Contêm uma string com o nível da trigger, sendo ROW ou STATEMENT
 - TG_OP: Contêm uma string indicando a operação, sendo INSERT, UPDATE, DELETE ou TRUNCATE
 - TG_RELID: Contêm o Object ID da tabela que causou a invocação da trigger
 - TG_TABLE_NAME: Nome da tabela que iniciou a invocação da trigger
 - TG_TABLE_SCHEMA: Nome do esquema da tabela que causou a invocação da trigger
 - TG_NARGS: Número de argumentos dado a função da trigger no CREATE TRIGGER
 - TG_ARGV[]: Array de strings com os argumentos do CREATE TRIGGER. O índice inicia de 0



Triggers

- 1) Criar uma trigger que verifique e grave o nome de novos clientes em MAIÚSCULO (função UPPER(varchar))
- 2) Crie uma nova tabela chamada "log", com os seguintes atributos: "identificador" (serial), "tabela" (varchar com 50 posições), "operacao" (varchar com 10 posições), "dadosNovos" (texto), "dadosAntigos" (texto);
- 3) Crie uma trigger de log para as tabelas a serem monitoradas via trigger são: "Hóspedes" e "Reservas"; A trigger deve fazer as seguintes operações:
 - a) Quando ocorrerem atualizações (UPDATES) nos registros dessas tabelas, o SGBD deverá inserir registros na tabela "log", preenchendo seus atributos com o nome da tabela que está sendo modificada, a operação que está sendo executada ("UPDATE") e o conteúdo anterior e atual dos registros que estão sendo modificados;
 - b) Quando ocorrerem exclusões (DELETES) de registros dessas tabelas, o SGBD deverá inserir registros na tabela "log", preenchendo seus atributos com o nome da tabela cujos registros estão sendo excluídos, a operação que está sendo executada ("DELETE") e o conteúdo dos registros que estão sendo excluídos.

